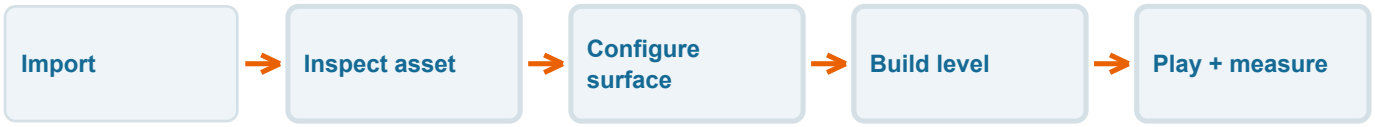
SCOPE / SYLLABUS 2.2-2.9

Import-to-game validation guide

Target: Unreal Engine 5.7, English UI, Windows. Static environment meshes and a conventional non-Nanite exercise path.

Required end result
A saved environment level that launches in a packaged Development build; correct asset scale and pivots; intentional materials, UVs and collision; controlled foliage; recorded texture and frame-time results.



Inputs and project setup

1. Prepare an FBX prop/module, a high/low comparison pair, a plant mesh, a UV-checker image, and their texture sets. Record intended dimensions and packed-map channel meanings.
2. In the project browser, choose **Games** → **Third Person** → **Blueprint** → **Create**. Keep the template gameplay actors and lighting for the exercise.
3. Open its playable map. Click **File** → **Save Current Level As**; save **L_AssetValidation** in **Content/Environment/Maps**.
4. Press **Ctrl+Space**. In the Content Browser, right-click → **New Folder**. Under **Environment**, create **Meshes, Materials, Textures, Foliage, Maps**.
5. Click **Play**. Confirm movement works, then press **Esc**. Use **File** → **Save All** after each completed stage.

Pages	Operation
2-4	Import, scale, names, material slots and pivot
5-9	Geometry, LODs, UVs and optional lightmaps
10-13	Repeatable materials, instances, normal and AO maps
14-17	Collision, placement and foliage
18-22	Textures, performance and packaged validation
23-26	Acceptance record, fault lookup and source register

Screenshot protocol. All engine images are genuine screenshots from Epic documentation. Each included control and arrow target was visually checked. They show reference assets and may retain an older UE5 layout. Follow the written target values; source screenshots are not a continuous capture of this project. Orange A/B/C arrows are added annotations. Diagrams are labeled separately.

2.2 / INTEGRATION

01 / Import the Static Mesh

Start in Content Browser → Environment/Meshes → Import → select the FBX → Open.



ENGINE REFERENCE. Interchange: A = category tabs; B = uniform scale; C = mesh type. Screenshot C currently reads None; set Static Mesh for this exercise. [S01]

1. In **Interchange Pipeline Configuration**, select **Default Assets Pipeline**. Turn off **Basic Layout** if required to expose advanced settings.
2. Under **Common**, set **Offset Uniform Scale = 1**. Under **Common Meshes**, set **Force All Mesh as Type = Static Mesh**. Preserve authored normals by leaving **Recompute Normals** off.
3. Open **Static Meshes**: enable import, disable **Combine Static Meshes** for separate reusable objects, enable collision import/generation, and disable **Build Nanite** for this exercise.
4. In **Materials** and **Textures**, disable automatic import. Configure these assets explicitly on pages 10-13. Click **Import**, then double-click the resulting mesh.

If the dialog says FBX Import Options	Equivalent choice
Skeletal Mesh	Off
Import Uniform Scale / Combine Meshes	1 / Off for separate assets
Normal Import Method	Import Normals; match tangent handling to the bake
Material Import Method / Import Textures	Do Not Create Material / Off

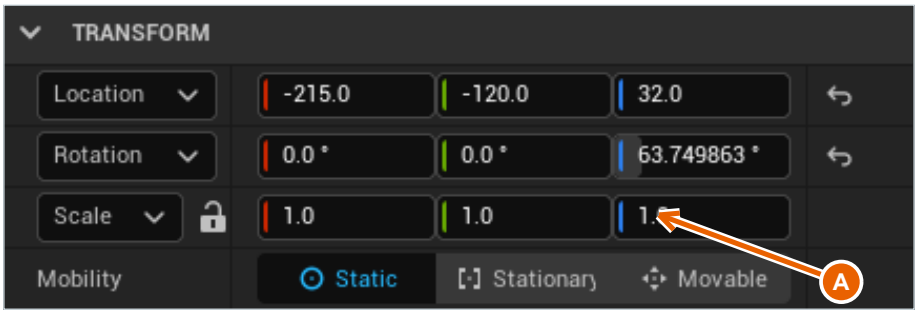
End result
The expected object exists as a Static Mesh, with correct orientation and no missing parts. Read import warnings before continuing.

References: S01 Interchange Import Reference | S02 FBX Import Options Reference

2.2 / SCALE AND NOMENCLATURE

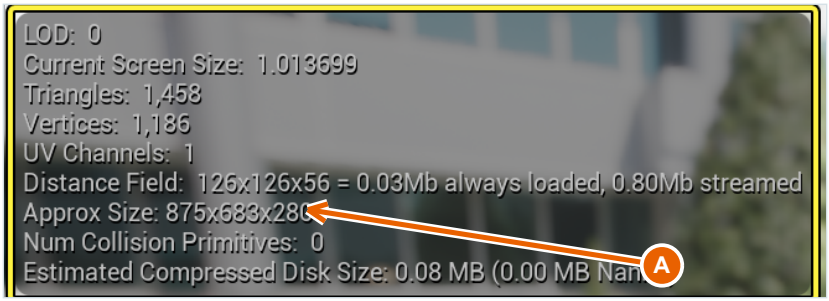
02 / Check scale and asset names

Asset dimensions and placed-actor scale are separate checks. Validate both.



ENGINE REFERENCE. A = actor Scale X/Y/Z. The shown values are 1.0. Location and rotation are example values. [S04]

1. Drag the mesh into the level. Select it and open **Details** → **Transform**. Set **Scale X/Y/Z = 1**.
2. Add a reference through **Quick Add (+)** → **Shapes** → **Cube**. A standard cube at scale 1 measures **100 × 100 × 100 cm**. Compare the mesh with this cube and the template character.
3. Double-click the mesh. Read **Approx Size** in the Static Mesh Editor viewport statistics. With default units, values are centimeters. A 3 m wall should be about 300 cm tall.
4. In the Content Browser, select the asset → **F2** → enter a consistent name. Rename inside Unreal so asset references can be updated.



ENGINE REFERENCE. A = mesh dimensions. This reference boulder is 875 × 683 × 280 cm; those are not target dimensions for your model. [S03]

Type	Classroom naming pattern
Static Mesh	SM_Wall_01
Material / instance	M_Surface_Master / MI_Wall_Stone
Textures	T_Wall_BC / T_Wall_N / T_Wall_AO

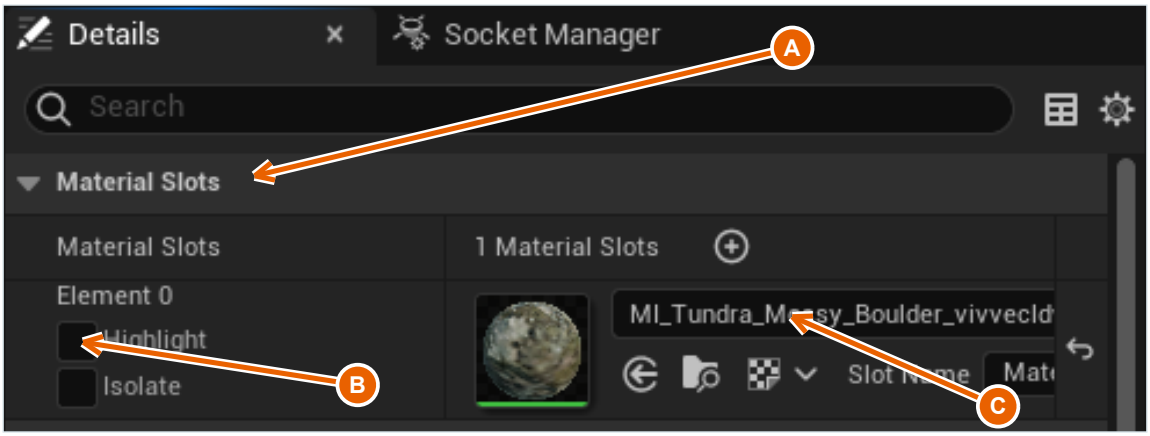
End result
Dimensions match the asset specification at actor scale 1. The name identifies asset type and purpose.

If scale fails
Correct source units/export transforms and reimport. A documented import-scale conversion is acceptable. Avoid compensating for every incorrect asset with a different actor scale.

2.2 / MATERIAL SPACES AND PIVOT

03 / Validate slots and pivot

A material slot assigns a material to a set of faces. The pivot determines placement and rotation behavior.



ENGINE REFERENCE. A = Material Slots; B = Highlight/Isolate for Element 0; C = material assignment. This asset has one slot. [S03]

1. Double-click the mesh → **Details** → **Material Slots**. Inspect each slot. Enable **Highlight** or **Isolate** to identify its faces. Check for empty or unintended duplicate slots.
2. If the wrong faces belong to a slot, correct their material IDs in the modeling application and reimport. Changing the assigned material does not change those face groups.
3. Return to the level, select the actor, press **W**, then **E**. Check that the gizmo is at a useful point and that rotation behaves as intended.
4. For an in-engine pivot correction, duplicate the Static Mesh asset first if its existing instances must stay unchanged. Select the duplicate actor → **Selection Mode dropdown** → **Modeling** → **XForm** → **Edit Pivot**.
5. Move the tool gizmo or use a **Box Positions** preset. Enable **Apply to All LODs** when appropriate, then click **Accept** and save. If Modeling is missing: **Edit** → **Plugins** → **Modeling Tools Editor Mode**; enable and restart.

Object	Useful pivot for this exercise
Wall / floor module	Snapping corner on the module grid
Tree / ground prop	Base at ground contact
Hinged door mesh	Hinge axis

End result
Only necessary material slots remain. Placement and rotation use a deliberate pivot.

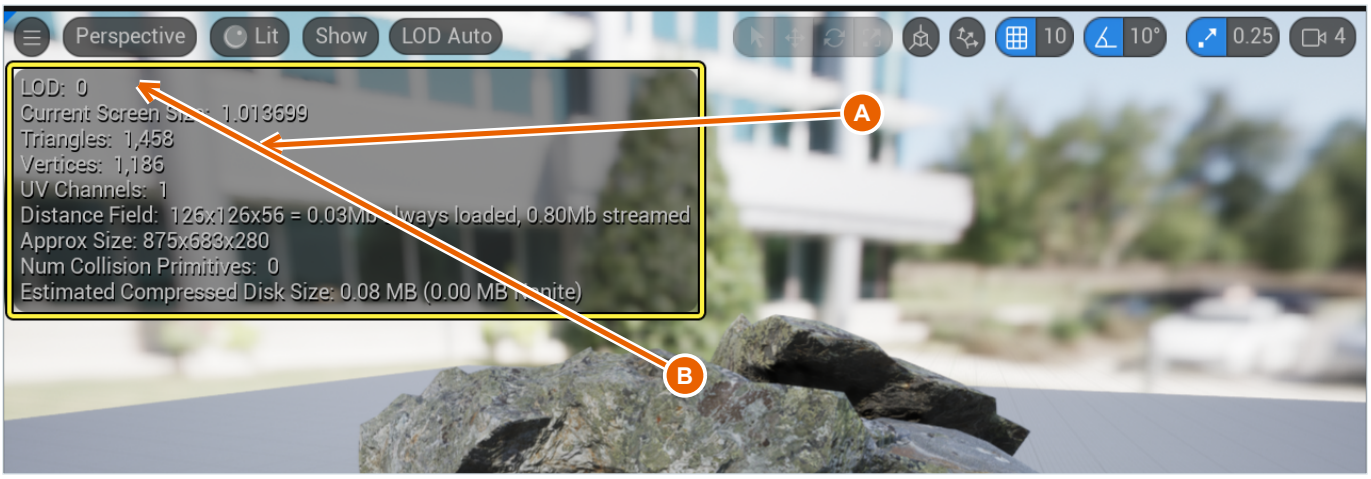
Scope of an edit
Edit Pivot changes the mesh asset and can affect every instance. A temporary actor pivot offset is not the same operation. Keep the source model consistent with the approved pivot for future reimports.

References: S05 Edit Pivot Tool

2.4 / HIGH-TO-LOW EVALUATION

04 / Compare high and low geometry

Use two separate meshes at equal world scale. Evaluate them from the intended gameplay camera distance.



ENGINE REFERENCE. A = triangles and vertices; B = active LOD. This is a statistics example, not a high/low pair. [S03]

1. Import the high and low versions as **SM_Prop_High** and **SM_Prop_Low**. Place them side by side at **Scale = 1**.
2. Open each mesh. Select **LOD 0** in the preview picker and record **Triangles** and **Vertices**. Compare the same LOD, not one mesh at LOD 0 and another at Auto.
3. Open the viewport **Lit** dropdown → **Wireframe**. Look for dense areas that do not affect silhouette or deformation. Return to **Lit**.
4. Inspect silhouette, thin parts, curved edges, hard edges and seams. Check both the closest playable view and a distant view. For a baked asset, validate the low mesh with its intended normal map.
5. Keep the high mesh in a comparison area or source archive. Remove its actor from the final runtime test unless it is deliberately part of that test.

Record	High mesh	Low mesh
LOD 0 triangles	_____	_____
LOD 0 vertices	_____	_____
Closest test distance	_____ cm	_____ cm
Visible silhouette/shading defects	_____	_____

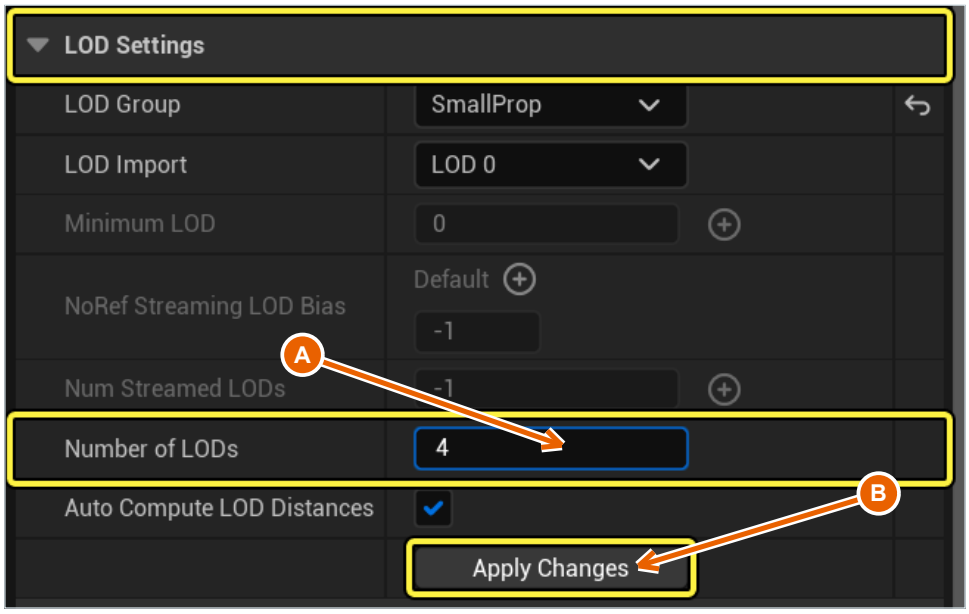
End result
The chosen runtime mesh preserves required silhouette and shading at the intended viewing distance. Its geometry cost is recorded.

High-to-low limitation
Importing a high and low mesh does not bake detail between them. Prepare the normal/AO bake in the authoring workflow. Normal maps change shading; they do not restore removed silhouette geometry.

2.4 / CONVENTIONAL GEOMETRY OPTIMIZATION

05 / Generate and inspect LODs

Use this exercise with Enable Nanite Support disabled. Nanite requires a separate platform-specific evaluation.



ENGINE REFERENCE. A = Number of LODs; B = Apply Changes. The screenshot uses 4; this exercise uses 3. [S06]

1. Open the runtime mesh → **Details** → **Nanite Settings**. Verify **Enable Nanite Support** is off.
2. Expand **LOD Settings**. Choose a suitable existing **LOD Group**, or configure manually. Set **Number of LODs = 3** and click **Apply Changes**.
3. Expand **LOD 1** → **Reduction Settings**. Test a triangle percentage, apply, and inspect the result. Repeat for LOD 2. Treat reductions as experiments, not fixed requirements.
4. In the viewport **LOD Auto** dropdown, preview **LOD 0**, **LOD 1** and **LOD 2** individually. Watch the silhouette and material boundaries.
5. Return to **LOD Auto**. Move the camera toward and away from the mesh. If transitions pop visibly, reduce the simplification or adjust **Screen Size** thresholds. Save.

Inspection	Pass condition
Triangle counts	Lower LODs reduce geometry usefully
Silhouette / shading	No unacceptable holes, collapsed thin parts or shading changes
Automatic transition	Changes are acceptable at the intended gameplay distance

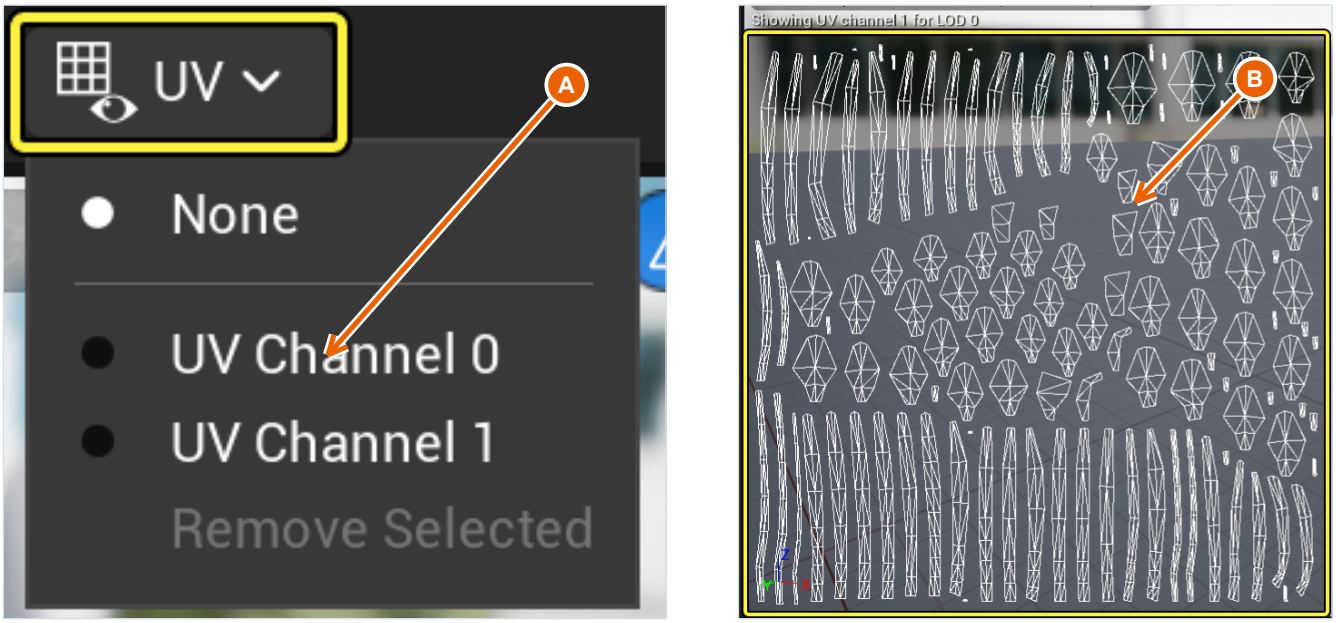
End result
The asset has a deliberate LOD chain and uses Auto selection during normal preview and gameplay.

Do not confuse the controls
Screen Size is screen coverage, not a distance in centimeters. A forced preview LOD verifies one mesh; it does not validate automatic transitions.

2.6 / CHANNELS, OVERLAP AND LAYOUT

06 / Inspect UV channels

UV channel indices begin at 0. Identify the purpose of every channel before editing or deleting it.



ENGINE REFERENCES. A = UV dropdown and channel selection; B = displayed UV layout. The right reference shows UV Channel 1 for LOD 0. [S07]

1. Double-click the mesh → toolbar **UV** dropdown → **UV Channel 0**. Inspect the displayed islands.
2. Switch through each available channel. Record which one drives surface textures and which, if any, is reserved for baked lighting.
3. Check for collapsed islands, unexpected mirrored areas, unnecessary overlap and layouts that do not match the supplied texture set.
4. To confirm material usage, open the material and inspect **TextureCoordinate** → **Coordinate Index**. Its value selects the UV channel used by connected texture samples.
5. Use the checker test on page 8 to assess distortion and physical texture scale; the UV overlay alone is insufficient.

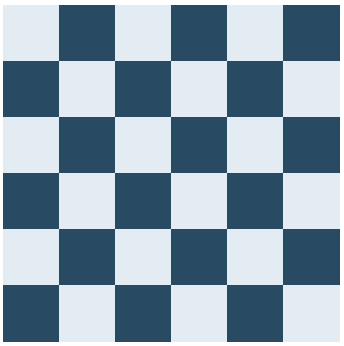
UV use	Overlap rule
Repeating or mirrored surface textures	Intentional overlap can be valid; judge it against the material/bake.
Unique baked normal/AO textures	Overlap must agree with the baking workflow; accidental stacking can create artifacts.
Lightmap UVs for baked lighting	Unique islands, inside 0-1, with padding and no overlaps.

End result
Every channel has an identified use. The material samples the intended channel.

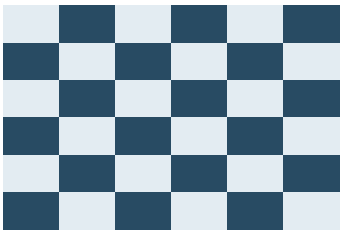
2.6 / DISTORTION AND CONSISTENCY

07 / Run the checker test

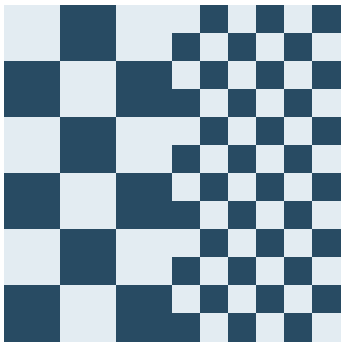
Use a square checker image with visible squares or numbers. Test neighboring assets at their intended world scale.



PASS: square checks



FAIL: stretched checks



FAIL: density mismatch

INSPECTION DIAGRAM. Compare checker shape and physical size on the mesh. These are schematic patterns.

1. Import the checker into **Environment/Textures**. In **Materials**, right-click → **Material** → name it **M_UV_Check** → double-click.
2. Drag the checker texture from the Content Browser into the material graph. Drag its **RGB output** → **Base Color** input on the material root. Click **Apply** → **Save**.
3. Select the actor in the level → **Details** → **Materials** → **Element**; assign **M_UV_Check** temporarily. Use an actor override so the asset default remains available.
4. Inspect the entire object. Squares should keep their shape except for justified distortion. Compare the physical checker size across neighboring assets.
5. For a uniform repetition adjustment, add **TextureCoordinate** and edit **U Tiling / V Tiling**, or use the parameter setup on page 10. This changes sampling; it does not repair the stored UV layout.
6. For stretched islands, accidental overlap or inconsistent unwrap scale, correct the source UVs and reimport. Re-run the test, then restore the production material override.

End result
No unacceptable stretched checks. Comparable surfaces use a consistent visible texture scale. The production material is restored afterward.

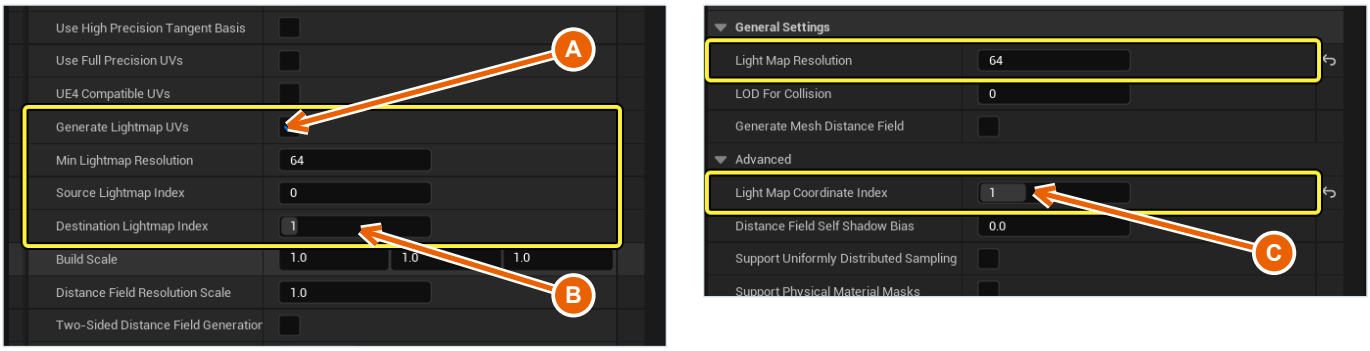
Correction boundary
A material tiling value can adjust repetition across an entire surface. It cannot independently fix badly shaped islands. Generating lightmap UVs is not a repair for surface texture distortion.

References: S07 Using UV Channels With Static Meshes | S10 Material Inputs

2.6 / OPTIONAL BAKED-LIGHTING BRANCH

08 / Configure lightmap UVs if needed

Skip this page for a fully dynamic-lighting exercise. A surface UV channel is still required for UV-mapped textures.



ENGINE REFERENCES. A = Generate Lightmap UVs; B = source/destination indices; C = Light Map Coordinate Index. Values shown are source 0 and destination 1. [S08]

1. Open the mesh → **LOD 0** → **Build Settings**. Enable **Generate Lightmap UVs**.
2. Set **Source Lightmap Index** to the existing usable texture layout, commonly **0**. Set **Destination Lightmap Index** to a separate channel, commonly **1**. Do not overwrite another required channel.
3. Set **Min Lightmap Resolution** for the intended minimum bake resolution, then click **Apply Changes**.
4. Open **General Settings** → **Advanced** → **Light Map Coordinate Index** and choose the generated destination. Set a suitable **Light Map Resolution** for the asset.
5. Use **UV** → **destination channel** to inspect packing. In a project configured for baked lighting, run **Build** → **Build Lighting Only**, then inspect seams and overlap warnings.

Control	Meaning
Min Lightmap Resolution	Packing/padding constraint used by the generator
Light Map Resolution	Resolution of the asset lightmap for the bake
Light Map Coordinate Index	UV channel used to store baked lighting

End result
The baked-lighting channel is separate, uniquely packed and correctly selected. A lighting build has no unresolved asset-related UV errors.

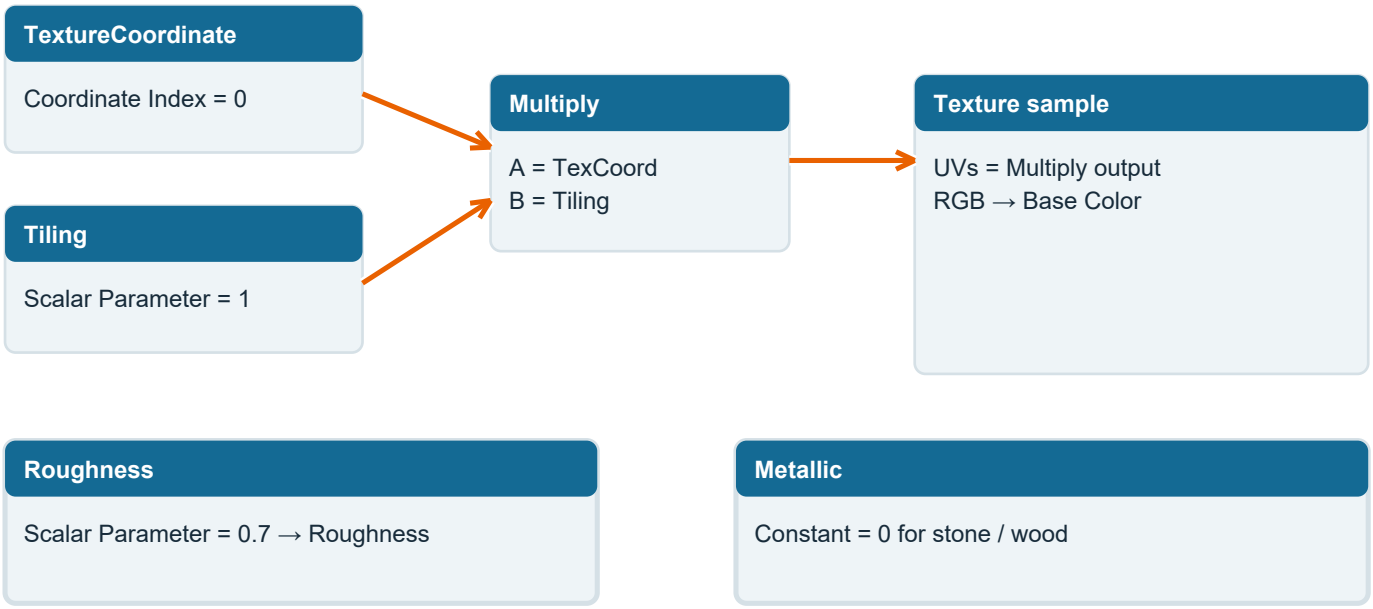
Common failure
Generation repacks existing charts; it does not create the missing seams needed to fix every unwrap. Lumen hides precomputed static-lighting contributions. Configure a baked-lighting project before assessing a bake.

References: S08 Generating Lightmap UVs | S16 Lumen Global Illumination and Reflections

2.7 / UV SCALE AND REUSABLE PARAMETERS

09 / Build the repeatable material

This graph uses the legacy-style Base Color / Roughness / Normal root inputs, supported in UE5.7 including Substrate-enabled projects.



CONNECTION DIAGRAM. Build these nodes in the Material Editor. This is a schematic, not an engine screenshot.

1. In **Environment/Materials**, right-click → **Material** → name it **M_Surface_Master** → double-click.
2. Select the material root. Use **Surface**, **Opaque** and **Default Lit** for this solid-surface exercise. Use a new unconverted material if the supplied graph already uses a Substrate slab.
3. Drag in the tileable base-color texture. Right-click its sample → **Convert to Parameter** → name it **BaseColorTexture**. Connect **RGB** → **Base Color**.
4. Right-click empty graph space and add **TextureCoordinate**, **Scalar Parameter** and **Multiply**. Name the scalar **Tiling**, set default **1**, and wire the UV branch shown above.
5. Add a scalar parameter **Roughness = 0.7** and connect it to Roughness. For stone or wood, connect a **Constant = 0** to Metallic. Click **Apply** → **Save**.

End result
A reusable parent material compiles successfully. Tiling changes the sample UVs; Roughness changes the surface response.

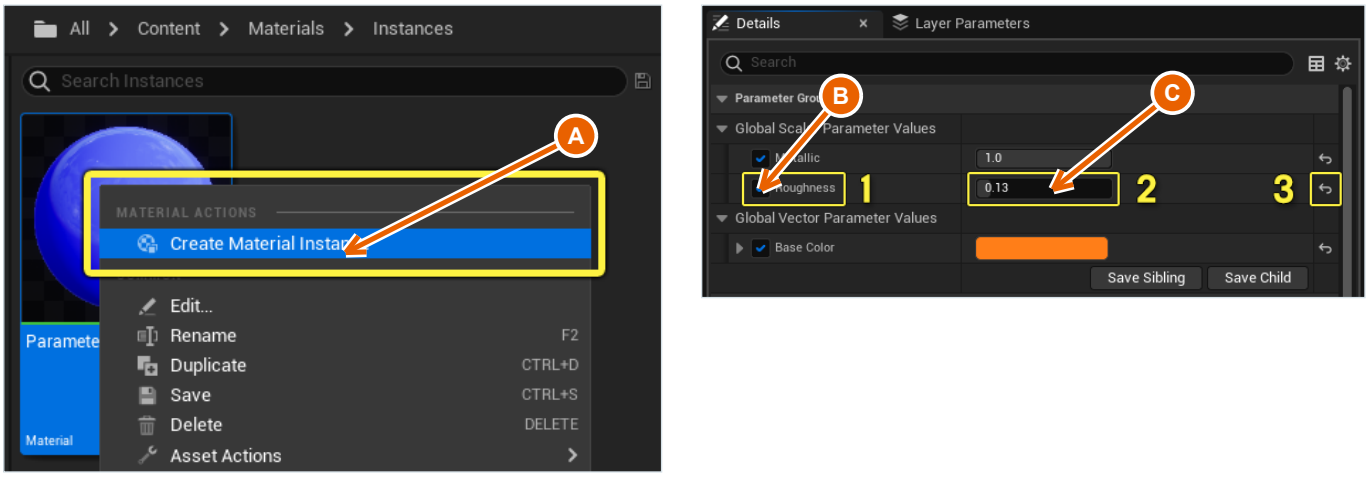
Mapping rule
Use repeating textures for this branch. Tiling a uniquely baked prop texture misaligns its details. When adding matching tileable normal/AO textures, connect the same UV output to all of them.

References: S09 Creating and Using Material Instances | S17 Substrate Materials Overview

2.7 / REUSABLE MATERIAL VARIATION

10 / Create and assign instances

The parent graph defines exposed parameters. An instance overrides selected values.



ENGINE REFERENCES. A = Create Material Instance; B = override checkbox; C = value field. Right image demonstrates Roughness = 0.13; use your specified values. [S09]

1. In the Content Browser, right-click **M_Surface_Master** → **Create Material Instance**. Name it **MI_Wall_Stone**.
2. Double-click the instance. Enable the checkbox beside **Tiling**, then enter **1**, **2** and **4** in turn. Compare the repetition rate.
3. Enable **Roughness**. Compare a low and high value between **0 and 1**. Save the approved value.
4. Open the Static Mesh → **Material Slots** → assign **MI_Wall_Stone** to the intended slot → save. This establishes the asset default.
5. Create a second instance from the same parent. Change a parameter and apply it to another actor through **Details** → **Materials** for a local variation.

Problem	Check
Parameter does not appear	The parent uses a parameter node; Apply and Save the parent.
Value cannot be changed	Enable the instance override checkbox.
Placed actor does not update	Check for an actor Materials override hiding the asset default.
All instances change unexpectedly	You edited the parent/default or the shared instance, rather than a separate variation.

End result
Two variations share a parent. Students can adjust exposed parameters without rebuilding the graph.

Material instances support reuse and organization. They do not, by themselves, combine all actors into one draw call.

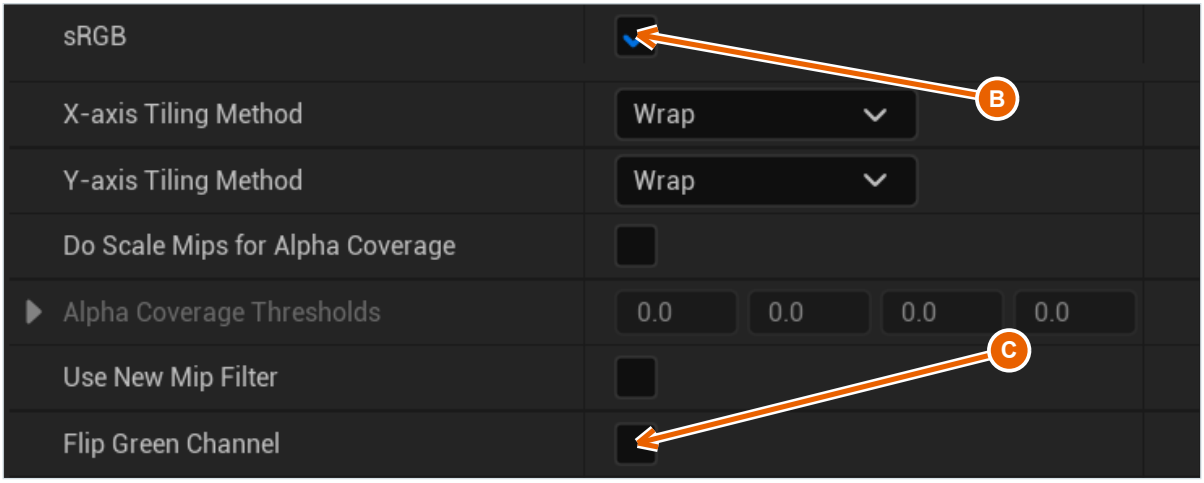
2.8 / TEXTURE SETTINGS AND SHADING

11 / Validate the normal map

Import the normal texture separately. Its suffix and blue/purple appearance are clues, not proof that its settings are correct.



ENGINE REFERENCE. A = Compression Settings. This general-texture reference shows Default; choose Normalmap for a normal map. [S11]



ENGINE REFERENCE. B = sRGB; C = Flip Green Channel. Disable sRGB for a normal map. Flip green only when the source convention requires it. [S11]

1. Double-click **T_Wall_N**. In **Details**, search **Compression Settings** and choose **Normalmap**. Verify **sRGB** is off; save.
2. Open the parent material. Add the normal texture sample, then connect **RGB → Normal**. Verify the sample node uses **Sampler Type = Normal**.
3. For a tileable texture set, connect the same scaled UVs used by base color. For a unique bake, use the matching unmodified bake UVs. Click **Apply → Save**.
4. Inspect at the closest gameplay distance under angled light. Compare the mapped result against a flat tangent-space normal: a **Constant3Vector (0,0,1)** connected directly to Normal.
5. If the source uses the opposite green/Y convention, open the texture and toggle **Flip Green Channel**. Compare with known raised and recessed details; keep the setting that matches the bake.

End result
Relief shades in the intended direction, without unexpected seams, inverted bumps or sampler errors. The normal map changes shading, not the mesh outline.

If flipping green does not fix it
Check UV channel, bake mesh, tangent basis, imported normals and triangulation. Do not use Two Sided as a general fix for flipped face normals. Correct the source geometry when faces are reversed.

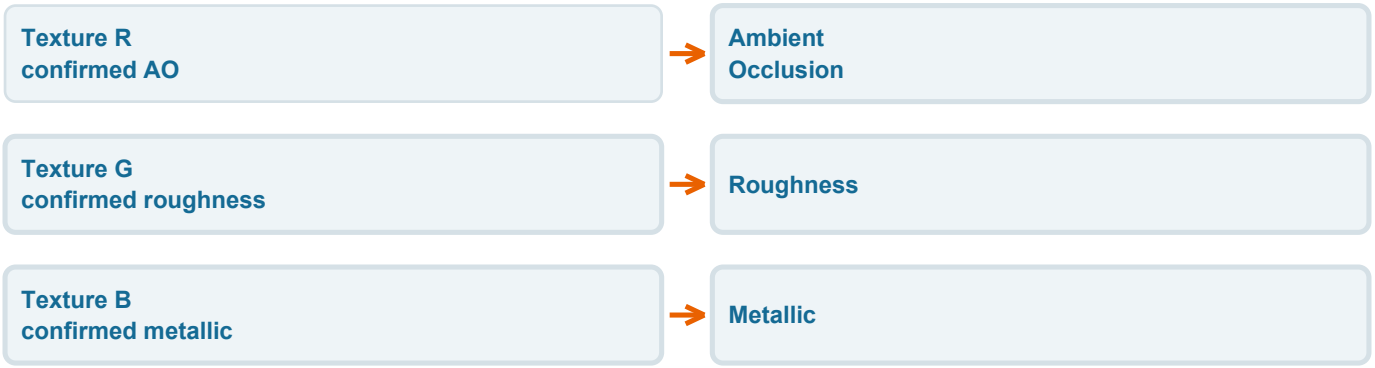
References: S11 Texture Asset Editor | S10 Material Inputs

2.8 / CHANNEL MEANING AND COMPARISON

12 / Validate AO and packed maps

AO is numeric data. Confirm the export preset before wiring a packed texture.

Texture	sRGB	Compression / connection
Base color	On, for ordinary color images	Default → RGB to Base Color
Tangent normal	Off	Normalmap → RGB to Normal
Standalone AO	Off	Masks for this exercise → R to Ambient Occlusion
Packed ORM, if confirmed	Off	Masks → R: AO, G: Roughness, B: Metallic



CONNECTION DIAGRAMS. ORM is an example packing convention. Other exports can put these maps in different channels.

1. Import and open the AO or packed texture. Set **Compression Settings = Masks** and verify **sRGB = off**. Save.
2. In the material, add the texture sample. Connect the confirmed channel outputs to the corresponding inputs shown above. Use the UVs required by that texture set.
3. For a controlled AO comparison, keep camera and lighting fixed. Compare **AO texture → Ambient Occlusion** against a **Constant = 1** connected to the same input. Apply each version before comparing.
4. If Lumen shows little or no change, check the material-AO path. Epic documents **Edit → Project Settings → Rendering → Allow Static Lighting = off** for Lumen material AO; restart if prompted. Use a dedicated dynamic-lighting exercise project for this change.

End result
Data textures use linear sampling and verified channels. AO is connected correctly and assessed under a compatible lighting configuration.

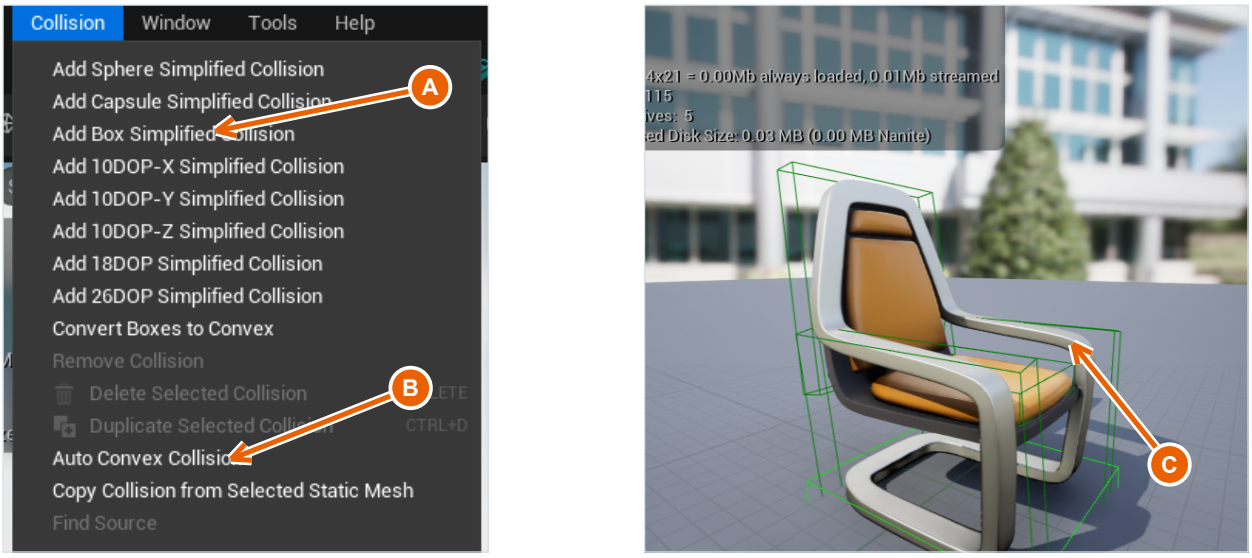
Common pitfalls
Black AO means occluded; white means unoccluded. Do not multiply AO into Base Color as a substitute for the AO input. A subtle result under direct light does not, alone, prove the connection is wrong.

References: S10 Material Inputs | S11 Texture Asset Editor | S16 Lumen Global Illumination and Reflections

GAME-READINESS / COLLISION

13 / Build and test collision

Collision must follow intended gameplay, including openings that remain traversable.



ENGINE REFERENCES. A = Add Box Simplified Collision; B = Auto Convex Collision. C = visible simple collision around a reference chair. [S13]

1. Open the Static Mesh Editor → viewport **Show** → **Simple Collision**. Inspect the existing shapes first.
2. For a simple solid prop, choose **Collision** → **Add Box Simplified Collision**. For a more complex solid shape, choose **Collision** → **Auto Convex Collision**, adjust the decomposition settings, and click **Apply**.
3. Inspect all sides. Remove unwanted or oversized hulls through the **Collision** menu and replace them with appropriate shapes. Save the mesh.
4. Select the actor in the level → **Details** → search **Collision Presets**. For a stationary solid training prop, choose **BlockAll**. Keep **Simulate Physics** off unless physics is required.
5. Click **Play**. Walk into the prop from several directions. Test stairs, doorways, arches, narrow passages and the top surface if it should support the player.

End result
The object blocks the player where intended, supports traversal correctly, and does not create invisible barriers across visible openings.

Common pitfalls
A single convex hull can fill an arch or doorway. Use multiple suitable hulls or authored custom collision. Do not apply Use Complex Collision As Simple to every asset as a blanket fix. Decorative grass may need no collision; a tree trunk usually does.

2.3 / PLACEMENT, REPETITION AND DISTRIBUTION

14 / Place and compose the level

Build with validated assets. Keep a playable route through the environment.



ENGINE REFERENCE. Toolbar detail from an older UE5 reference layout. A = translation grid snap; B = rotation snap. Use the matching icons in your viewport. [S04]

1. Drag the main wall/floor modules from the Content Browser into the level. Use **W / E / R** for move, rotate and scale.
2. Enable **translation grid snapping** and choose an increment that matches the module specification, for example **10 cm**. Enable rotation snapping with a compatible angle, for example **15°**.
3. Select an actor and hold **Alt** while dragging a translation gizmo axis to duplicate it. Check seams at floor and eye height.
4. Place large structures first, then medium props, then small details. Preserve modular dimensions; rotate organic props and vary group spacing where appropriate.
5. In the **Outliner**, right-click → **Create Folder** to organize structures, props and vegetation. Press **G** for Game View to hide editor icons.
6. Click **Play**. Walk from the entrance to the focal area. Check visibility, route width, camera clearance and repeated patterns. Save.

Check in the player view	Required result
Module seams	No unintended cracks, intersections or floating edges
Route	Character and camera can traverse the intended path
Composition	The destination/focal area is readable from chosen test views
Repetition	No distracting repeated orientation, spacing or texture landmarks
Organization	Outliner folders and asset names identify their purpose

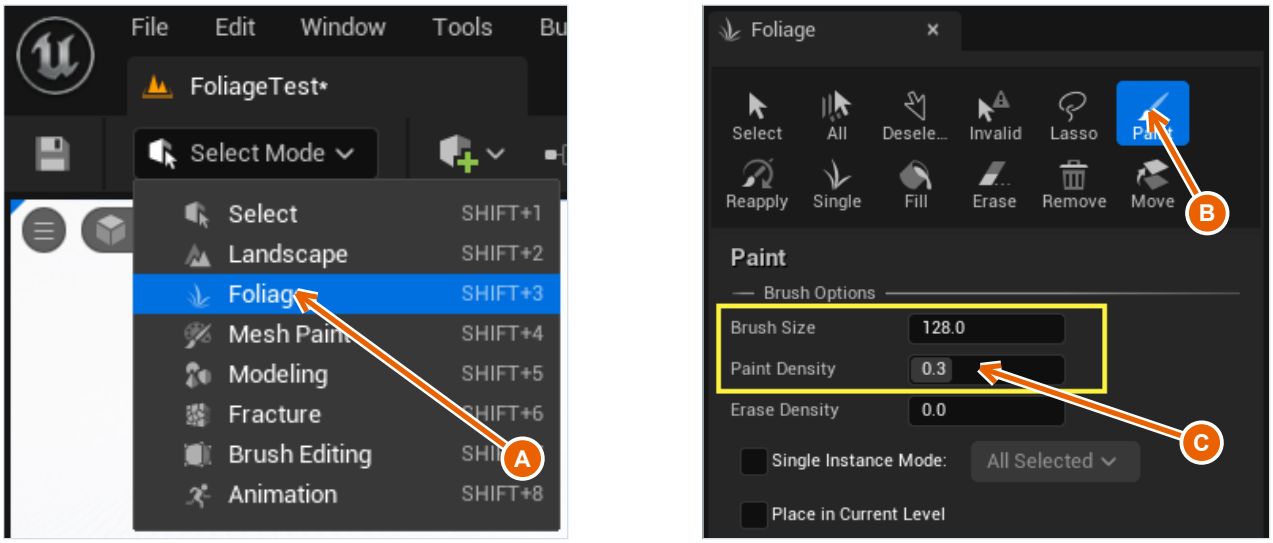
End result
A coherent playable layout with aligned modules and deliberate asset distribution.

Duplicating actors reuses asset data, but ordinary duplicated actors are not automatically guaranteed to render in one draw call.

2.5 / FOLIAGE INTEGRATION

15 / Add and paint foliage

Use a validated conventional plant mesh with its intended material before painting many instances.



ENGINE REFERENCES. A = Foliage mode; B = Paint; C = Paint Density. The brush screenshot uses 0.3; use the exercise starting values below. [S12]

1. Open the **Selection Mode dropdown** → **Foliage**, or press **Shift+3**.
2. Open the Content Drawer. Drag your plant **Static Mesh** into the **Foliage Palette / Mesh List**. Hover over its entry and enable its checkbox.
3. Select **Paint**. Under surface filters, enable **Landscape** for terrain, or **Static Meshes** for a mesh floor. Disable surfaces that should not receive plants.
4. For a first small patch, try **Brush Size = 200** and **Paint Density = 0.1**. These are test values, not production density targets.
5. Point at the ground and paint with the left mouse button. Hold **Shift** while painting to erase. Set **Erase Density = 0** for complete removal in the brushed area.
6. Add a second plant type and paint a separate test patch. Keep the path and required sight lines clear.

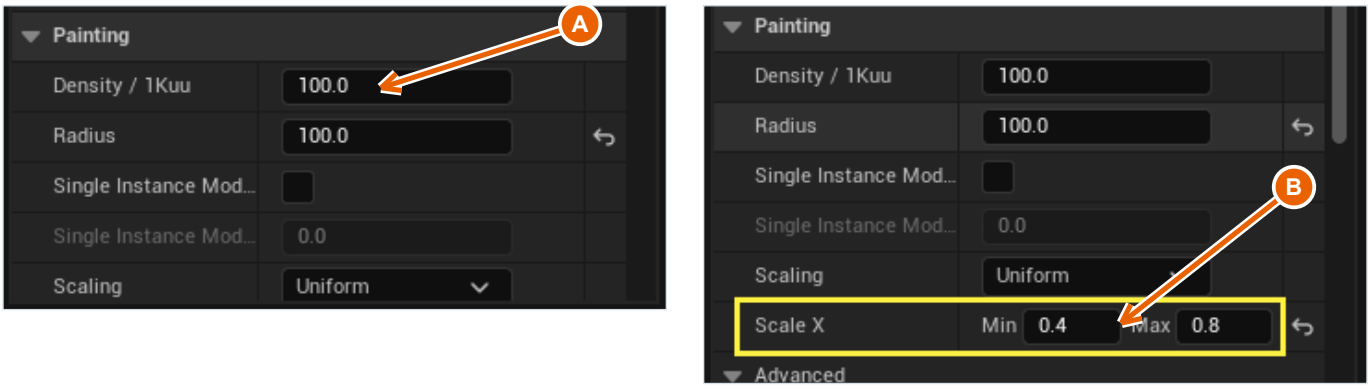
End result
Selected plant types appear only on the intended ground surfaces, sit at believable ground contact, and do not obstruct the player route.

If nothing paints
Check the foliage entry checkbox, Paint tool, surface filters, ground collision, slope/height limits, radius and density. Paint Density is a multiplier of each foliage type's density.

2.5 / DENSITY, SCALE AND CULLING

16 / Control variety and distance cost

Inspect a small patch before populating the full level. Settings changes do not always update existing instances automatically.



ENGINE REFERENCES. Painting settings details: A = type Density / 1Kuu; B = uniform Scale X range. Select your plant entry before changing these fields. [S12]

1. Click the plant entry. Under **Painting**, adjust **Density / 1Kuu** and **Radius**. Radius controls spacing; high density cannot overcome restrictive spacing.
2. Use **Scaling = Uniform**. For moderate test variation, set **Scale X Min = 0.85** and **Max = 1.15**. Enable **Random Yaw**; use **Align to Normal** only when the plant should follow the surface slope.
3. For existing painted instances, select **Reapply**, enable the checkboxes for the settings to update, then brush over the affected patch. Repaint a small area if necessary.
4. Find **Cull Distance** in the type settings. Set a nonzero End distance, then walk toward and away from the patch. For example, test **End = 5000 cm**, then adjust for the camera and plant size.
5. If a fade is needed for masked foliage, connect **PerInstanceFadeAmount × existing opacity mask → Opacity Mask**, and set Start below End. Retest shadows and disappearance in Play.

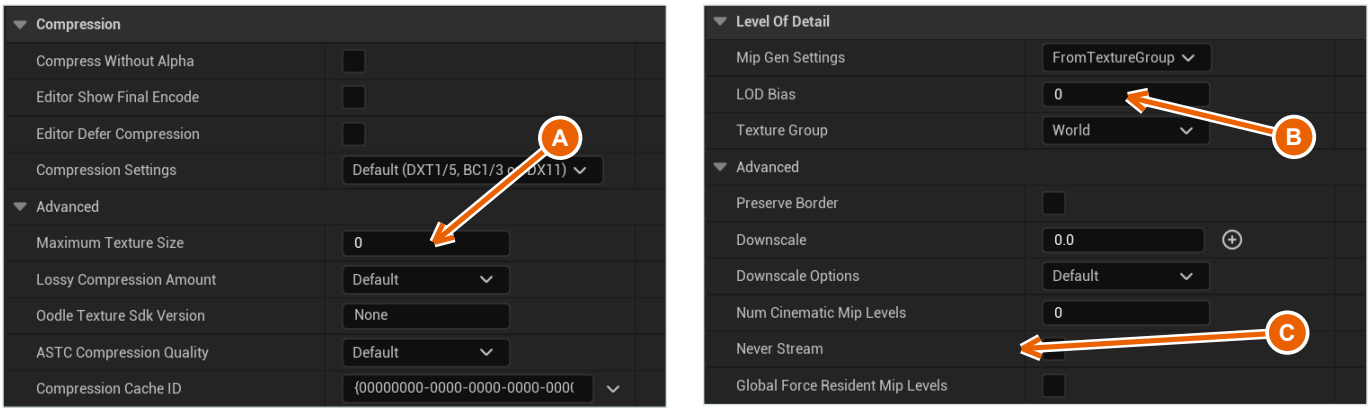
End result
Density and variation support the scene. Distance removal/fading is acceptable and improves the measured cost.

Scope and pitfalls
This distance-culling exercise uses non-Nanite foliage. Nanite meshes do not follow the same foliage cull-distance/fade behavior. Static Mesh Foliage uses instancing, but materials, masked overdraw, shadows and wind still contribute cost.

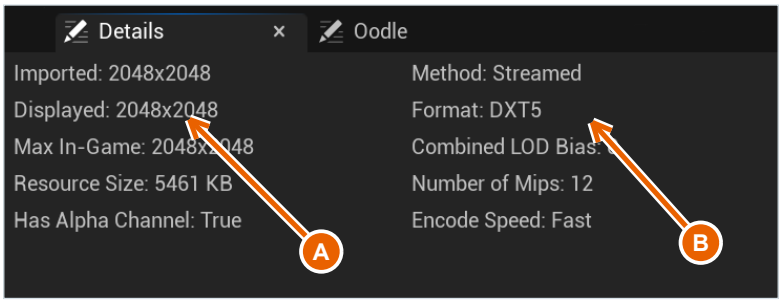
2.9 / RESOLUTION, MIPMAPS AND STREAMING

17 / Optimize texture resolution

Compare at the closest intended gameplay distance. Do not judge only from the texture preview.



ENGINE REFERENCES. A = Maximum Texture Size; B = Texture Group; C = Never Stream. The screenshot Maximum Texture Size value 0 means no explicit per-asset cap. [S11]



ENGINE REFERENCE. A = Max In-Game size; B = Number of Mips. Imported, displayed and in-game sizes can differ. [S11]

1. Double-click the texture. Record **Imported** size, **Max In-Game**, **Resource Size**, **Number of Mips** and streaming Method.
2. Set a relevant **Texture Group**, such as **World** or **WorldNormalMap**. Retain mipmaps and leave **Never Stream** unchecked for ordinary streamable environment textures.
3. For a selected 4096 texture, test **Maximum Texture Size = 2048**. Save and inspect the asset in Play at the fixed test distance. Try 1024 only if necessary detail remains acceptable.
4. Choose the lowest tested resolution that meets the visual requirement. Check normal, roughness and AO detail separately; they may not require the same resolution as base color.

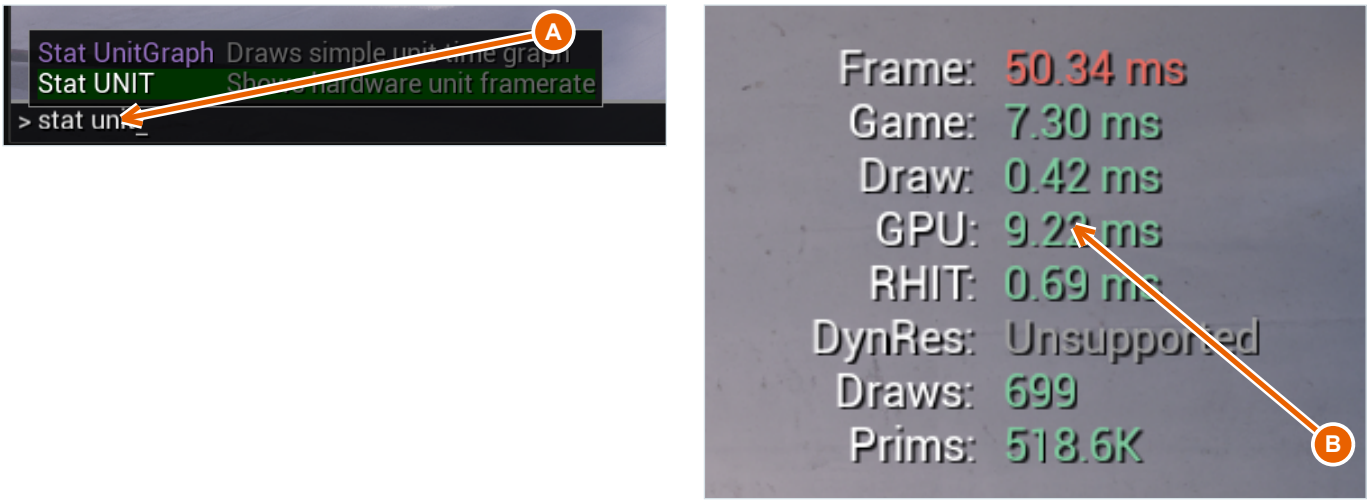
End result
Texture resolution is justified by the gameplay view. Mipmaps, streaming, compression and color space are appropriate.

Do not infer total GPU memory from one field
Resource Size is an asset resource estimate, not the level's complete VRAM use or package size. Max In-Game is a ceiling; streaming may currently load a smaller mip.

2.9 / REPRODUCIBLE MEASUREMENT

18 / Establish the performance baseline

Capture the template/early layout first, the dressed level second, and the optimized level third.



ENGINE REFERENCES. Two detail crops from the same runtime screenshot. A = stat unit command; B = time counters. These values are an example, not a target. [S14]

1. Record hardware, resolution, render method, **Settings → Engine Scalability Settings** preset, screen percentage/upscaler, VSync and frame limit. Keep them fixed for comparisons.
2. Click **Play** and open the console, usually **~**. If the keyboard layout differs, check **Edit → Project Settings → Input → Console Keys**.
3. Enter **stat fps**, **stat unit** and **stat streaming** separately. Close the console to view the scene. Enter the same stat command again to toggle that overlay off.
4. Wait for compilation/loading to settle. Sample the same stationary view, then walk the same route. Record typical frame times and recurring spikes.
5. Change one asset setting, repeat the route and compare. Confirm the conclusion again in the packaged Development build.

Metric	Interpretation
Frame	Overall frame interval; 60 FPS is approximately 16.7 ms.
Game / Draw / GPU	Initial clues about CPU game work, render work and GPU time. They overlap; do not add them.
Texture streaming	Check pool pressure and missing/late texture detail while moving.

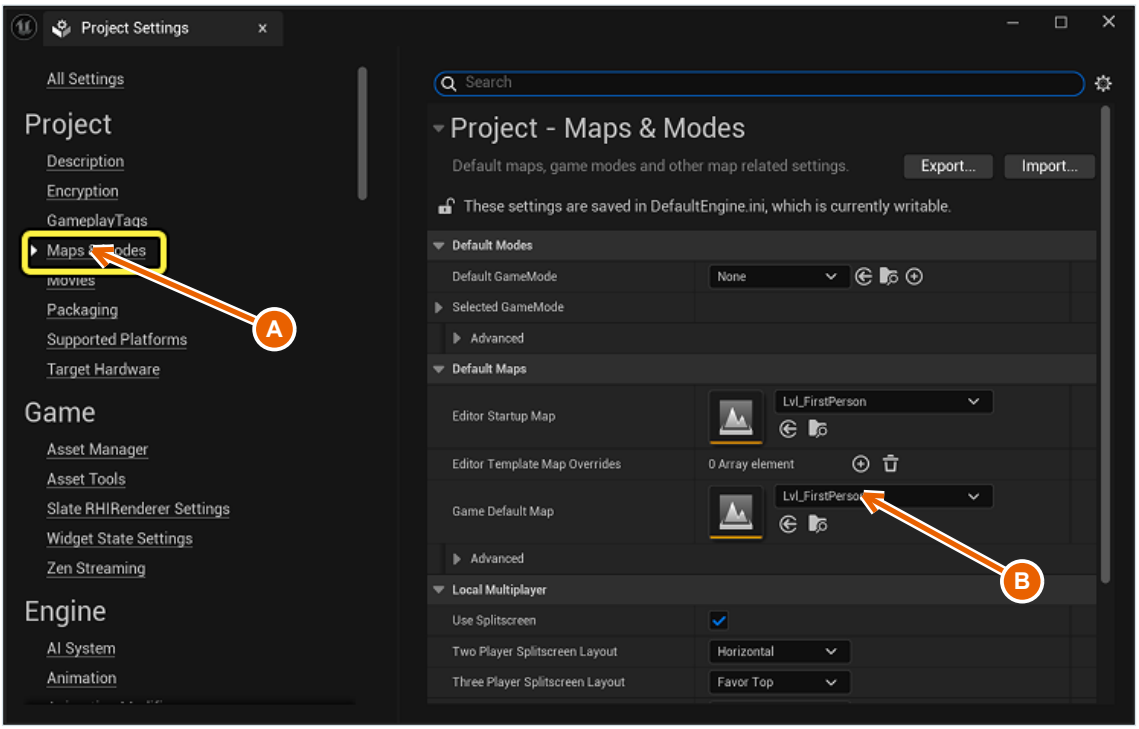
End result
A repeatable before/after record shows both visual quality and frame-time effects. Editor measurements are an initial baseline, not final shipping proof.

A frame limiter or VSync can inflate Frame relative to the visible work counters. These overlays indicate where to investigate; they are not a complete bottleneck diagnosis.

GAME-READINESS / BUILD CONFIGURATION

19 / Select the playable startup map

A packaged build opens the Game Default Map, which can differ from the map currently open in the editor.



ENGINE REFERENCE. A = Maps & Modes; B = Game Default Map. The screenshot selects Lvl_FirstPerson; choose L_AssetValidation in your project. [S15]

1. Use **File → Save All**. Open **Edit → Project Settings → Maps & Modes**.
2. Under **Default Maps**, set **Game Default Map = L_AssetValidation**. Editor Startup Map is optional and does not replace this setting.
3. Check **Default GameMode** or the map's **Window → World Settings → GameMode Override**. The Third Person template game mode must spawn the playable character.
4. Confirm a usable **Player Start** exists above valid ground and is not obstructed by collision. Test Play before packaging.
5. In **Project Settings → Packaging**, select **Build Configuration = Development** for this validation build. If using an explicit maps-to-cook list, include the validation map.

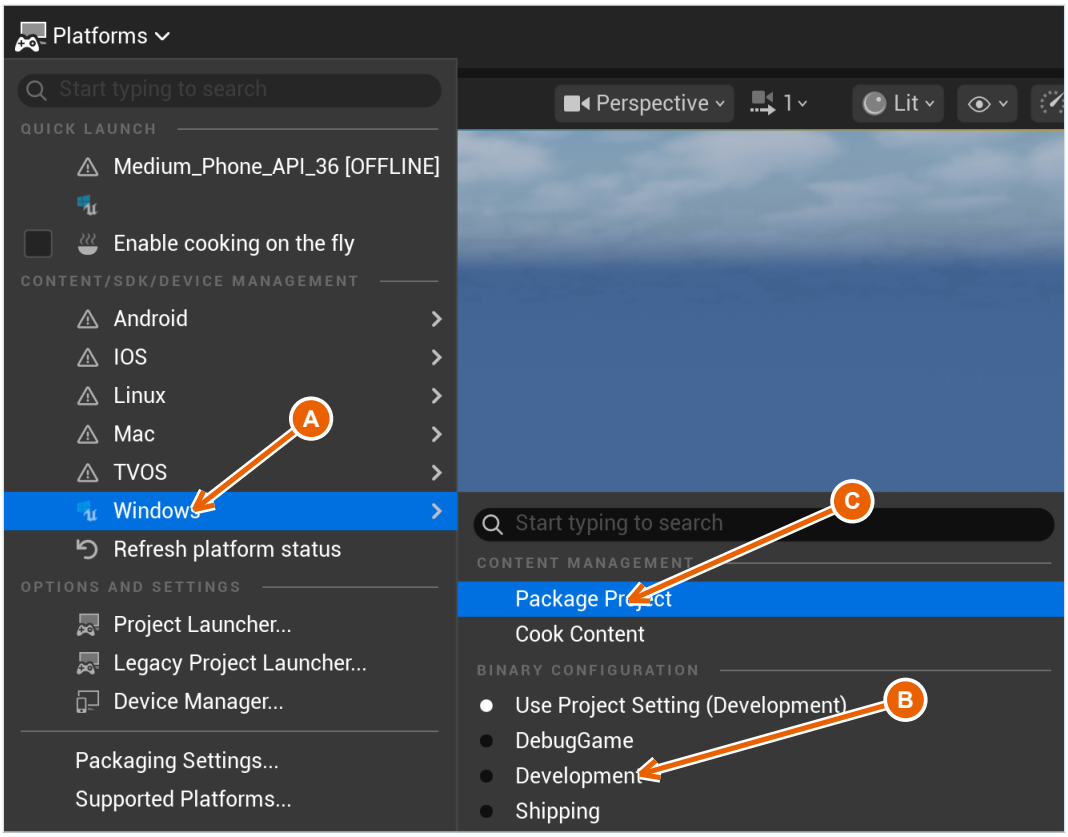
End result
The selected startup map, game mode and player start form a working playable entry point.

Common failure
A correct asset can still appear in a black or non-playable packaged level when the wrong map, game mode, pawn or spawn point is configured. Check these before changing mesh/material settings.

GAME-READINESS / WINDOWS BUILD

20 / Package the Development build

Use a packaged build to verify runtime loading and gameplay outside the editor.



ENGINE REFERENCE. A = Windows; B = Development option; C = Package Project. Use Project Setting (Development) is also valid when the project setting is Development. [S15]

1. In the Level Editor toolbar, open **Platforms** → **Windows**. Select **Development**, or **Use Project Setting (Development)** when already configured.
2. Choose **Package Project**. Select a separate output directory, for example **Builds/Windows_Validation**, and wait for completion.
3. If packaging fails, click **Show Output Log** in the packaging notification. Find the actual error and its referenced asset/toolchain; a final AutomationTool failure line alone is not the cause.
4. If Windows reports a missing SDK/toolchain, resolve the required installed-engine platform support and build tools before retrying. This is a build-environment prerequisite.

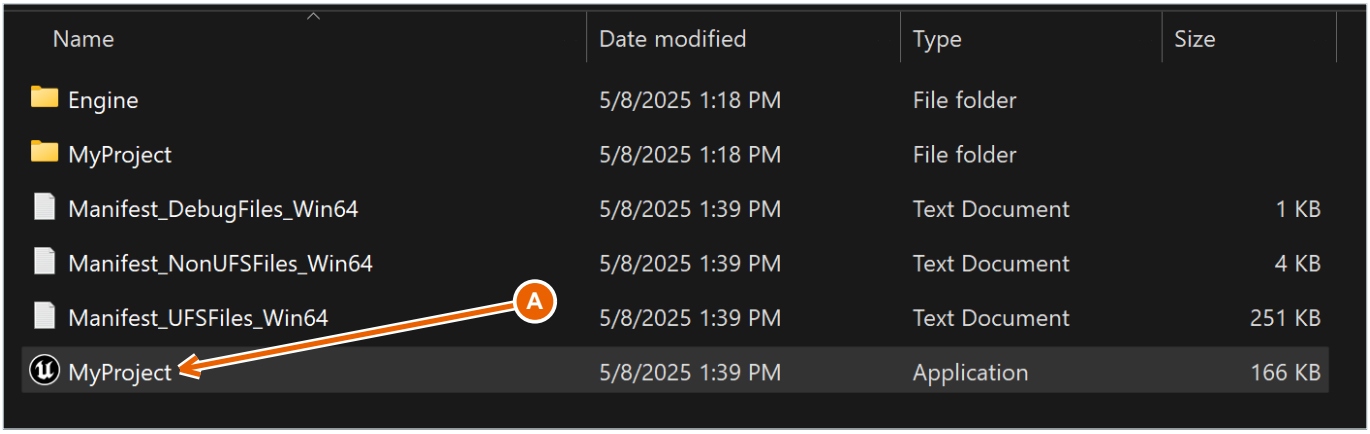
End result
Packaging completes successfully and creates a Windows output folder containing a runnable application and its supporting files.

Common failure
Cook Content alone does not produce the same runnable package. Keep all packaged output folders with the executable; copying only the .exe is insufficient.

GAME-READINESS / FINAL PLAYTEST

21 / Validate the packaged result

Launch the packaged executable on the intended test hardware. Repeat the same route and quality settings used for the baseline.



ENGINE REFERENCE. A = packaged application. This is Epic's Windows Explorer example, not an engine-editor panel. Your application uses your project name. [S15]

1. Open the package output folder and double-click the project application. Confirm it starts in **L_AssetValidation**.
2. Walk the full route; turn around, approach props, cross doorways and move through the foliage area. Revisit the closest planned inspection views.
3. In the Development build, use the console/stat overlays where enabled. Repeat the frame-time and streaming sample under the same conditions.
4. Record failures, correct the source asset or project setting, package again and repeat the affected checks. Do not accept an editor-only fix without rebuilding.

System	Required visible/functional result
Scale and placement	Character scale is believable; modules align; no floating props.
Surfaces	Expected material instances and textures load; no default checker/error material.
Geometry and UVs	No unacceptable silhouette changes, stretched textures or seams at test views.
Collision	Solid objects block correctly; doorways and routes remain traversable.
Foliage	Ground contact, density, distance behavior and collision match intent.
Performance	Frame times and streaming meet the defined hardware/resolution target.

End result
A packaged, playable environment passes the recorded visual, functional and performance criteria. Packaging success alone does not certify the asset.

FINAL DELIVERABLE / PASS-FAIL EVIDENCE

22 / Complete the acceptance record

Mark P (Pass) or F (Fail) beside each topic. An unresolved required failure blocks acceptance.

Topic	Acceptance requirement	Evidence to retain
2.2	Correct real dimensions at actor scale 1; consistent names; useful pivot; intentional material slots.	Dimension readout + pivot/slot capture
2.3	Aligned placement; readable route; controlled repetition and distribution.	Player-view images from fixed positions
2.4	Runtime mesh retains required silhouette/shading; LOD behavior is acceptable.	High/low counts + same-distance comparison
2.5	Foliage has appropriate spacing, variation, ground contact and distance behavior.	Paint/type settings + near/far views
2.6	UV channels have known roles; checker test passes; baked branch passes if used.	UV overlay + checker screenshots
2.7	Parent material compiles; instances expose and apply useful parameters.	Parent graph + two instance variants
2.8	Normal/AO configuration and channel connections match the authored texture set.	Texture settings + controlled comparison
2.9	Resolution decisions and measured performance changes are documented.	Before/after log under fixed settings
Runtime	Packaged build launches, loads assets and passes traversal/collision checks.	Build folder + test result record

Minimum measurement record

Condition	Before optimization	After optimization
Hardware / resolution / quality	_____	_____
Frame / Game / Draw / GPU ms	_____	_____
Streaming warning / visible loading	_____	_____
Change made / visual tradeoff	_____	_____

Required final state
Approved asset settings + saved playable level + packaged Development build + evidence record. A screenshot of a good-looking object alone is insufficient.

TROUBLESHOOTING / RE-TEST AFTER CHANGES

Fault lookup and correction order

Diagnose the failure before adding global overrides. Reimporting can affect normals, slots, UVs and collision; inspect them again.

Observed failure	Check / correction
Object is 100× too small/large	Check source units, FBX transforms and import conversion. Reimport and compare Approx Size at actor scale 1.
Rotation happens around a distant point	Check the mesh pivot. Correct in the source or use Modeling → XForm → Edit Pivot on the intended asset.
Faces are missing from one side	Check source face normals and whether the surface is intentionally one-sided. Correct reversed faces before using Two Sided.
Material is assigned to wrong faces	Inspect slot Highlight/Isolate. Fix source material IDs; changing a slot material does not regroup faces.
Texture stretches or changes scale	Check checker shape, UV channel, actor scale and material tiling. Repair distorted islands in the source.
Normal bumps look inverted	Confirm Normalmap compression, linear sampling and Normal sampler type. Compare green-channel convention and tangent basis.
AO does not visibly change the object	Verify channel contents, linear sampling and material input. Check lighting-path support and compare under controlled indirect lighting.
Player falls through / hits invisible wall	Inspect simple collision and actor collision preset. Replace missing/oversized hulls; test all traversable openings.
Foliage will not paint	Check selected-type checkbox, surface filter, ground collision, density, radius and slope/height limits.
Foliage disappears abruptly	For conventional meshes, test End Cull Distance and material-based fading. Recheck Nanite status and shadows.
Textures blur or stream late	Check Max In-Game, mip/streaming settings and stat streaming. Reduce unnecessary demand before increasing memory budgets.
Frame time is high	Keep test conditions fixed. Compare Game, Draw, GPU and limiter behavior; test one likely contributor at a time.
Package opens wrong/black map	Check Game Default Map, maps-to-cook list, game mode, player start and loading/build errors.
Editor fix is absent in the package	Save changes, rebuild the package, launch the new output, and repeat the affected validation.

Re-test rule
After reimport: dimensions → pivot → normals → UV channels → material slots → collision → LODs → Play → packaged result. Keep the approved source files with the evidence record.

REFERENCE / VERIFIED CONTROL CORRESPONDENCE

Source and screenshot register

References were retrieved 27 October 2026. Documentation is requested with application_version=5.7; Epic retains some screenshots from earlier UE5 layouts.

Every embedded screenshot was visually inspected. The register identifies the actual visible control used by the instruction. Screenshot asset names and sample values remain unchanged. Orange arrows and captions are PDF annotations. Pages 1, 8, 10, 13, 23 and 24 use text/tables or labeled diagrams rather than simulated engine screenshots.

Guide page	Verified screenshot content	Source
2	Interchange category tabs, Offset Uniform Scale and Force All Mesh as Type.	S01
3	Actor Scale X/Y/Z; mesh Approx Size readout.	S04 / S03
4	Material Slots, Highlight/Isolate, material picker. Pivot correction is text-only.	S03 / S05
5	LOD index, triangle count and vertex count.	S03
6	Number of LODs and Apply Changes.	S06
7	UV dropdown with channels 0 and 1; displayed channel-1 layout.	S07
9	Generate Lightmap UVs, source/destination indices and Light Map Coordinate Index.	S08
11	Create Material Instance; override checkbox and Roughness field.	S09
12	Compression Settings, sRGB and Flip Green Channel.	S11
14	Add Box Simplified Collision, Auto Convex Collision and visible simple hulls.	S13
15	Translation and rotation snapping icons in an older UE5 toolbar.	S04
16	Foliage mode, Paint, Paint Density.	S12
17	Type Density / 1Kuu and uniform Scale X Min/Max.	S12
18	Maximum Texture Size, Texture Group, Never Stream, Max In-Game and Number of Mips.	S11
19	stat unit command and runtime time counters.	S14
20	Maps & Modes and Game Default Map.	S15
21	Windows platform submenu, Development and Package Project.	S15
22	Packaged application in Windows Explorer; explicitly labeled outside-editor image.	S15

Verification scope: control-to-instruction correspondence. The reference scenes are not claimed to be this exercise, and this PDF does not claim that the complete workflow was executed on the reader's project or hardware.

REFERENCE / TECHNICAL SOURCES

Epic documentation links

Click a source title to open the full technical reference. Engine screenshots are reproduced from these official Epic pages; arrows and diagrams were added for this guide.

S01 Interchange Import Reference

S02 FBX Import Options Reference

S03 Static Mesh Editor UI

S04 Transforming Actors

S05 Edit Pivot Tool

S06 Static Mesh Automatic LOD Generation

S07 Using UV Channels With Static Meshes

S08 Generating Lightmap UVs

S09 Creating and Using Material Instances

S10 Material Inputs

S11 Texture Asset Editor

S12 Foliage Mode

S13 Setting Up Collisions With Static Meshes

S14 Stat Commands

S15 Packaging Your Project

S16 Lumen Global Illumination and Reflections

S17 Substrate Materials Overview

S18 Recommended Asset Naming Conventions

Asset naming examples, inspection conditions, test values, acceptance criteria and measurement forms are classroom specifications. Numeric examples must be adjusted to the actual asset, camera, target hardware and project requirements.